TCSS 343 – Design and Analysis of Algorithms

Fibonacci Numbers

Fibonacci Numbers

- The famous Italian mathematician Leonardo Fibonacci is best known for the sequence of numbers that bears his name.

$$0,1,1,2,3,5,8,13,21, \ldots$$



  - Chapter 8 intro in the text.
  - Fibonacci
  - Fibonacci_number
  - Golden_ratio

- This Fibonacci series is formed by adding the last two numbers together to get the next number.
- This naturally recursive sequence is easily described using a self reduction.
- Let $F(n)$ be the nth Fibonacci number.

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

Fibonacci Numbers (Recursive)

- Let $F(n)$ be the nth Fibonacci number.

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

```
Fibonacci(n)
        If n = 0 return 0.
        If n = 1 return 1.
        Return Fibonacci(n − 1) + Fibonacci(n − 2)
End Fibonacci
```

- Can we express the runtime of this recursive algorithm as a recurrence?

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

- This recurrence is not of the right form for the Master Theorem so we will have to revisit our backup tools of guess and prove by induction.

Fibonacci Numbers (Recursive)

$$T(n) = \begin{cases} c & \text{if } n \leq 1 \\ T(n-1) + T(n-2) + 1 & \text{if } n > 1 \end{cases}$$

- Exercise: What is the shape of the recursive tree?
- Let's try to guess the runtime using some helpful approximations.

$$T_u(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T_u(n-1) + 1 & \text{if } n > 1 \end{cases}$$

$$T_l(n) = \begin{cases} c & \text{if } n \leq 1 \\ 2T_l(n-2) + 1 & \text{if } n > 1 \end{cases}$$

$$T_u(n) < T(n) < T_l(n)$$

- Exercise: Use recursion trees or repeated sums to come up with guesses for $T_u(n)$ and $T_l(n)$.

$$T_u(n) \in \Theta(2^n)$$
$$T_l(n) \in \Theta\left(2^{\frac{n}{2}}\right)$$

- What can we conclude from these bounds?
  - $T(n) \in \Theta(d^n)$ for some $\sqrt{2} < d < 2$.
  - In fact $T(n) \in \Theta(d^n)$ for $d = \phi = \frac{1+\sqrt{5}}{2} = 1.618\ldots$
  - This value is also known as the golden ratio and is the limit of the ratio between consecutive Fibonacci numbers.

Dynamic Programming

- We say a problem has optimal substructure when we can find a self-reduction from the problem to its sub-problems.
  - Whenever we have optimal substructure we can design a divide and conquer algorithm for the problem.
  - Like the Fibonacci problem explored here this solution is not always optimal.
- An optimal substructure has overlapping sub-problems when the same sub-problem is solved recursively more than once.
  - In the Fibonacci problem we had to recursively solve the same problem many times leading to an exponential number of recursive calls.
  - Is there a better way to calculate the Fibonacci sequence?

Dynamic Programming

```
Fibonacci(n)
        Let F[0 ... n] be a list of length n + 1.
        F[0] = 0
        F[1] = 1
        For i from 2 to n do:
                F[i] = F[i − 1] + F[i − 2]
        End For
        Return F[n]
End Fibonacci
```

- Example: $n = 14$:

0	1	1	2	3	5	8	13	21	34	55	89	144	233	377

Dynamic Programming

```
Fibonacci(n)
        Let F[0 … n] be a list of length n + 1.
        F[0] = 0
        F[1] = 1
        For i from 2 to n do:
                F[i] = F[i − 1] + F[i − 2]
        End For
        Return F[n]
End Fibonacci
```

- What is the runtime of this algorithm?
  - $O(n)$

- In Dynamic Programming we store the solutions to the sub-problems in an appropriate data structure and then retrieve them when needed instead of recursing.
  - This means that we will avoid the overlapping sub-problems.

Dynamic Programming

- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Notice that the Dynamic Programming design strategy begins exactly the same way as Divide and Conquer.
  - However, after we identify overlapping sub-problems we know the top-down recursive solution will be inefficient.
  - So we instead design a bottom-up iterative solution.

TCSS 343 – Design and Analysis of Algorithms

1-Dimensional

Dynamic Programming

Dynamic Programming

- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Additional resources on Dynamic Programming:
  - Dynamic_programming
  - Optimal_substructure
  - Overlapping_subproblems

Job Selection

- Job Selection Problem
  - You are a wealthy socialite who charges for public appearances. This year you have offers for appearances every day with different pay outs. The problem is they are all in different cities in North America so if you work one job you cannot work jobs in the day before or after due to travel. You want to select the jobs to work to make the most money this year.

- Can we express this problem formally?

Job Selection (JOB)
Input: $P[1 \dots n]$ a list of n positive numbers
Output: m a maximum possible payout and
$A \subseteq \{1, \dots, n\}$ a set of indexes such that
if $i \in A$ then $i + 1 \notin A$ and such that $m = \sum_{i \in A} P[i]$

- This is an example of an optimization problem.

Definition: An optimization problem is a problem where the goal is to find an optimum (i.e. a minimum cost or maximum reward).
  - An optimization problem typically has two components to the output:
    1. The value of the optimum (i.e. a numerical value).
    2. A means of achieving the optimum (this is usually problem specific).

- In dynamic programming we focus on finding the numerical component and then recover the rest of the solution afterwards.

Job Selection

Job Selection (JOB)
Input: $P[1 \ldots n]$ a list of n positive numbers
Output: m a maximum possible payout and
$A \subseteq \{1, \ldots, n\}$ a set of indexes such that
if $i \in A$ then $i + 1 \notin A$ and such that $m = \sum_{i \in A} P[i]$

- Let $JS(n)$ express the output for the Job Selection problem on input $P[1 \ldots n]$. Can we state a self-reduction from this problem to its sub-problems?

$$JS(n) = \begin{cases} 0 & \text{if } n = 0 \\ P[1] & \text{if } n = 1 \\ \max\{JS(n-2) + P[n], JS(n-1)\} & \text{if } n \geq 2 \end{cases}$$

- Notice that this self-reduction reduces the problem to sub-problems of size $n - 1$ and $n - 2$. These are the same sub-problems as the Fibonacci sequence.
  - We have a self-reduction so we have identified optimal sub-structure.
  - We have the Fibonacci pattern so we have overlapping sub-problems.
  - Thus – dynamic programming to the rescue!

Choosing a Data Structure

$$JS(n) = \begin{cases} 0 & \text{if } n = 0 \\ P[1] & \text{if } n = 1 \\ \max\{JS(n-2) + P[n], JS(n-1)\} & \text{if } n \geq 2 \end{cases}$$

- Now that we have decided that dynamic programming is the appropriate design strategy we have a few choices to make.
  - What data structure should we use to store the solutions to the sub-problems?
  - What order should we solve the sub-problems?

- To select a data structure we look at the dimension of our sub-problems.
  - In this problem, like Fibonacci, we use one variable to indicate the sub-problem. In both cases we called this n.
  - So we have a 1-dimensional problem.
  - We could create an array $J[1 \ldots n]$ to store the solutions to the sub-problems $1 \ldots n$.

- What order should we evaluate the problems?
  - In a 1-dimensional problem the choice is quite clear. We will start with the smallest problem and work our way up to the target problem.
  - We will initialize $J[1]$ and $J[2]$ and then use the self-reduction to fill in the remaining entries.

Dynamic Programming Algorithm

$$JS(n) = \begin{cases} 0 & \text{if } n = 0 \\ P[1] & \text{if } n = 1 \\ \max\{JS(n-2) + P[n], JS(n-1)\} & \text{if } n \geq 2 \end{cases}$$

```
JobSelection(P[1 ... n])
        Let J[0 ... n] be an array of numbers of size n + 1.
        Let J[0] = 0.
        Let J[1] = P[1].
        For i = 2 to n do:
                Let a = J[i − 2] + P[i].
                Let b = j[i − 1].
                Let J[i] = max{a, b}.
        End For
        Return J[n].
End JobSelection
```

Dynamic Programming Algorithm

```
JobSelection(P[1 ... n])
        Let J[0 ... n] be an array of numbers of size n + 1.
        Let J[0] = 0.
        Let J[1] = P[1].
        For i = 2 to n do:
                Let a = J[i − 2] + P[i].
                Let b = j[i − 1].
                Let J[i] = max{a, b}.
        End For
        Return J[n].
End JobSelection
```

P

15	3	5	12	6	6	8	13	9	4	8	7	2	3

J

0	15	15	20	27	27	33	35	46	46	50	54	57	57	60

Recovering the Solution

- We know the maximum earnings is 60 but we do not know which days to work to earn that much.
  - Can we determine this from our two lists?

P

15	3	5	12	6	6	8	13	9	4	8	7	2	3

J

0	15	15	20	27	27	33	35	46	46	50	54	57	57	60

- We can always store our choices in a tertiary array.
  - At each step we chose one of two possible sub-problems.
  - We can store a 1 if we choose to work day i and we store a 0 if we choose not to work day i.

S

1	0	1	1	0	1	1	1	0	1	1	1	0	1



- How can we recover the solution from this list?
  - $[14, 12, 10, 8, 6, 4, 1]$

Dynamic Programming Algorithm

```
JobSelection(P[1 ... n])
        Let J[0 ... n] be an array of numbers of size n + 1.
        Let S[1 ... n] be an array of bits of size n.
        Let J[0] = 0.
        Let J[1] = P[1].
        Let S[1] = 1.
        For i = 2 to n do:
                Let a = J[i − 2] + P[i].
                Let b = j[i − 1].
                Let J[i] = max{a, b}.
                If J[i] = a then S[i] = 1 else S[i] = 0.
        End For
        Return (J[n], Recover(S)).
End JobSelection

Recover(S[1 ... n])
        Let A be an empty set.
        Let i = n.
        While i > 0:
                If S[i] = 1 then
                        Add i to A.
                        Let i = i − 2.
                else let i = i − 1.
        End While
        Return A.
End Recover
```

TCSS 343 – Design and Analysis of Algorithms

Longest Increasing Subsequence

Longest Increasing Subsequence

- A sequence is an ordered set.
  - A subsequence is created from a sequence by deleting some elements.
  - A sequence $S = (s_1, s_2, \dots, s_n)$ is called increasing if $s_1 < s_2 < \cdots < s_n$.

- Given a sequence of integers can we find the longest increasing subsequence?

```
Longest Increasing Subsequence (LIS)
Input: S[1 ... n] a sequence of integers
Output: l the length of the longest increasing subsequence and
        A ⊆ {1,2, ..., n} such that |A| = l and if i, j ∈ A and i < j then S[i] < S[j]
```

- What is the longest increasing subsequence of this sequence?

$$S = [8,1,3,7,5,2,6,4,9,0]$$

$$[1,3,5,6,9]$$

Longest Increasing Subsequence

Longest Increasing Subsequence (LIS)
Input: $S[1 \dots n]$ a sequence of integers
Output: l the length of the longest increasing subsequence and $A \subseteq \{1,2, \dots, n\}$ such that $|A| = l$ and if $i, j \in A$ and $i < j$ then $S[i] < S[j]$

- We note that element $S[n]$ can be added to an earlier computed sequence that ends with $S[i]$ only if $S[n] > S[i]$.
- Let $LIS(n)$ express the longest increasing subsequence that includes S_n if run on input $S[1 \dots n]$.

$$LIS(n) = \max_{1 \le i < n} \{LIS(i) : S[n] > S[i]\} + 1$$

- For this self-reduction we assume that the max operator returns 0 if called with no arguments (and so we need no terminating case).

Longest Increasing Subsequence

$$LIS(n) = \max_{1 \le i < n} \{LIS(i) : S[n] > S[i]\} + 1$$

- Can we turn this self-reduction into a dynamic programming algorithm?

```
LongestIncreasingSubsequence(S[1 ... n])
        Let L[1 ... n] be an array of numbers of size n.
        For i = 1 to n do:
                Let L[i] = max_{1≤j<i} {L[j] : S[i] > S[j]} + 1
        End For
        Return max_{1≤i≤n} {L[i]}
End LongestIncreasingSubsequence
```

- What is the runtime of this algorithm?

$$\Theta(n^2)$$

Longest Increasing Subsequence

```
LongestIncreasingSubsequence(S[1 ... n])
        Let L[1 ... n] be an array of numbers of size n.
        For i = 1 to n do:
                Let L[i] = max_{1≤j<i} {L[i]: S[i] > S[j]} + 1
        End For
        Return max_{1≤i≤n} {L[i]}
End LongestIncreasingSubsequence
```

$$S = [8,1,3,7,5,2,6,4,9,0]$$

8	1	3	7	5	2	6	4	9	0
1	1	2	3	3	2	4	3	5	1

TCSS 343 – Design and Analysis of Algorithms

2-Dimensional

Dynamic Programming

Dynamic Programming

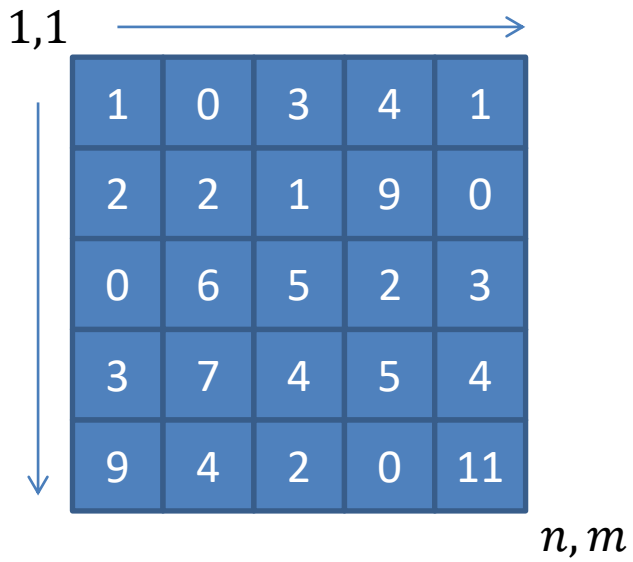
- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Additional resources on Dynamic Programming:
  - Dynamic_programming
  - Optimal_substructure
  - Overlapping_subproblems

Dungeon Escape

- You are a brave warrior trapped in the top left room (room $1,1$) of an $n \times m$ grid of rooms. The exit from the dungeon is in the bottom right room (room n, m). In every room there is some number of goblins $g(i,j) \geq 0$. You want to plan your escape so as to meet the minimum number of goblins. You decide in each step to only move closer to your goal so you will only move down or to the right.

- Can we express this problem formally?

  Dungeon Escape (DE)
  Input: $G[1 \ldots n][1 \ldots m]$ the number of goblins in each room
  Output: g a minimum number of goblins encountered and A a sequence of directions $\downarrow, \rightarrow$ indicating the optimal path to the exit



Dungeon Escape

- Can we express this problem formally?

  Dungeon Escape (DE)
  Input: $G[1 \dots n][1 \dots m]$ the number of goblins in each room
  Output: g a minimum number of goblins encountered and A a sequence of directions $\downarrow, \rightarrow$ indicating the optimal path to the exit

- What sub-problems should we consider?
  - We should consider the paths to cells to above and to the left of the current cell.
  - So for each problem we consider two sub-problems.

- Is every path from from cell $1,1$ to cell n, m the same length?
  - Yes. Length $n + m - 2$ moves.

- How many such paths are there?
  - Every path is a permutation of $n - 1$ of the characters $\downarrow$ and $m - 1$ of the characters $\rightarrow$.
  - How many such permutations are there?

$$\binom{n+m-2}{n-1}$$

Dungeon Escape

Dungeon Escape (DE)
Input: $G[1 \dots n][1 \dots m]$ the number of goblins in each room
Output: g a minimum number of goblins encountered and A a sequence of directions $\downarrow, \rightarrow$ indicating the optimal path to the exit

- Let $DE(n, m)$ express the output of the Dungeon Escape problem on input $G[1 \dots n][1 \dots m]$.

- Can we find a self-reduction from this problem to its sub-problems?

$$DE(n,m) = \begin{cases} G[1,1] & if\ n = 1\ and\ m = 1 \\ \min\{DE(n-1,m), DE(n,m-1)\} + G[n,m] & if\ n > 1\ or\ m > 1 \end{cases}$$

- Are there overlapping sub-problems in this self-reduction?
  - Yes.
- What data structure would be appropriate to store the solutions?
  - A 2d-array.
- What order should we solve the sub-problems?
  - Row by row: From row 1 to n. From column 1 to m.
  - We can also go column but column in order of row.

Dungeon Escape

$$DE(G[n,m]) = \begin{cases} G[1,1] & if\ n = 1\ and\ m = 1 \\ \min\{DE(G[n-1,m]), DE(G[n,m-1])\} + G[n,m] & if\ n > 1\ or\ m > 1 \end{cases}$$

- Can we turn this self-reduction into a dynamic programming algorithm?

```
DungeonEscape(G[1 ... n][1 ... m])
        Let P[1 ... n][1 ... m] be a 2d-array of numbers.
        Let P[1,1] = G[1,1].
        For i = 1 to n do:
                For j = 1 to m do:
                        Let P[i,j] = min{P[i − 1,j], P[i,j − 1]} + G[i,j].
                End For
        End For
        Return P[n,m].
End DungeonEscape
```

- What is the runtime of this algorithm?

$$O(nm)$$

Dungeon Escape

```
DungeonEscape(G[1 … n][1 … m])
        Let P[1 … n][1 … m] be a 2d-array of numbers.
        Let P[1,1] = G[1,1].
        For i = 1 to n do:
                For j = 1 to m do:
                        Let P[i, j] = min{P[i − 1, j], P[i, j − 1]} + G[i, j].
                End For
        End For
        Return P[n, m].
End DungeonEscape
```

1	0	3	4	1
2	2	1	9	0
0	6	5	2	3
3	7	4	5	4
9	4	2	0	11

1	1	4	8	9
3	3	4	13	9
3	9	9	11	12
6	13	13	16	16
15	17	15	15	26

	←	←	←	←
↑	↑	←	←	↑
↑	↑	↑	←	↑
↑	←	↑	↑	↑
↑	↑	↑	←	←

Dungeon Escape

```
DungeonEscape(G[1 … n][1 … m])
        Let P[1 … n][1 … m] be a 2d-array of numbers.
        Let D[1 … n][1 … m] be a 2d-array of numbers.
        Let P[1,1] = G[1,1].
        For i = 1 to n do:
                For j = 1 to m do:
                        Let P[i,j] = min{P[i − 1,j], P[i,j − 1]} + G[i,j].
                        If P[i,j] = P[i − 1,j] + G[i,j]
                                then D[i,j] = ↑
                                else D[i,j] = ←.
                End For
        End For
        Return (P[n,m], Recover(D)).
End DungeonEscape


Recover(D[1 … n][1 … m])
        Let S be an empty stack.
        Let i = n and j = m.
        While i > 1 and j > 1:
                If D[i,j] = ↑
                        then push ↓ onto S and i = i − 1
                        else push → onto S and j = j − 1
        End While
        Return S.
End Recover
```